LAPORAN PRAKTIKUM

PEMROGRAMAN BERORIENTASI OBJEK

PEKAN 6

Disusun Oleh :

Rahmat Ananda Nazar

2511532008

Dosen Pengampu : Afdhal Dinilhak, M.Kom

Asisten Praktikum : Muhammad Dawi Syauqi

Praktikum PBO : Kelas-B



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

1. IDENTITAS

Nama: Rahmat Ananda Nazar

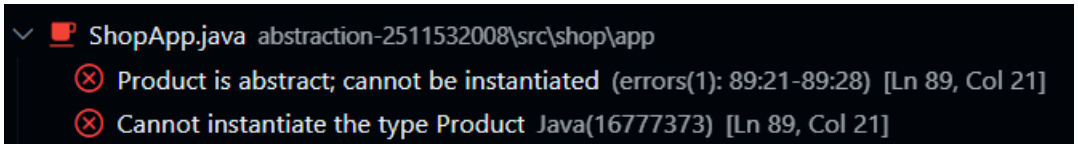
NIM: 2511532008

Kelas praktikum : B

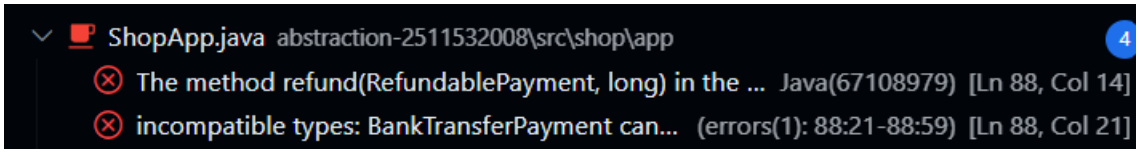
```
Parameter tugas untuk NIM 2511532008
  CARD_ADMIN_PERCENT  = 3
  TRANSFER_ADMIN_FEE  = 4000
  SMS_MAX_LENGTH      = 108
PS D:\Praktikum_PBO\abstraction-2511532008>
```

2. Lampiran Error Compiler

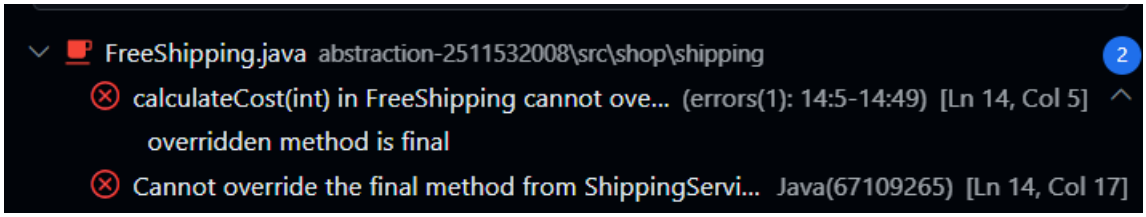
L1 (Compiler menolak pembuatan objek Product karena Product adalah abstract class. Abstract class hanya berfungsi sebagai cetakan bersama, sehingga objeknya harus dibuat dari subclass konkret seperti PhysicalProduct atau DigitalProduct)



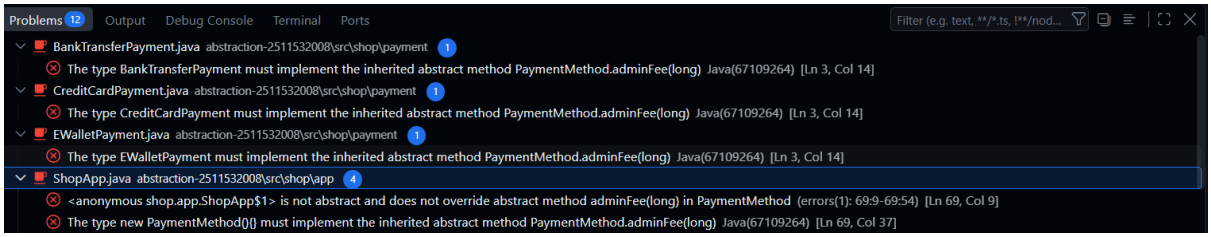
L3 (Method refund hanya menerima parameter bertipe RefundablePayment, sedangkan BankTransferPayment hanya mengimplementasikan PaymentMethod. Compiler menolak refund via transfer bank sebelum program dijalankan, tanpa memerlukan satu pun if pengecekan.)



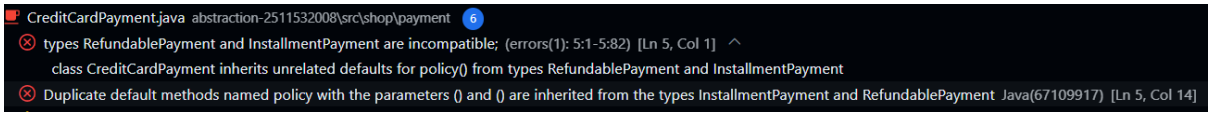
L4 (calculateCost() pada ShippingService bersifat final, sehingga subclass tidak boleh menggantinya. Dengan begitu rumus ongkir (berat dibulatkan ke atas per kilogram, lalu dikali tarif) terkunci untuk semua kurir, sedangkan tarif tetap bebas diatur subclass lewat ratePerKg().)



T.a . (Terjadi 4 error, yaitu pada EWalletPayment, BankTransferPayment, CreditCardPayment, dan anonymous class voucher di ShopApp. Anonymous class ikut rusak karena ia tetap sebuah class konkret yang mengimplementasikan PaymentMethod langsung, sehingga wajib mengisi semua method abstract, termasuk adminFee yang baru ditambahkan)



T.e (CreditCardPayment mengimplementasikan dua interface yang sama-sama punya default method policy() dengan signature identik. Java tidak tahu implementasi mana yang harus dipakai, sehingga compiler menolak dan memaksa programmer menyelesaikannya secara eksplisit dengan meng-override policy() di CreditCardPayment.)



3. Jawaban pertanyaan T1 dan T2, diagram UML T2, tabel T3, dan tabel T4

3a T1.

1. Mengapa EWalletPayment tidak perlu meng-override policy()?

EWalletPayment hanya mengimplementasikan satu interface yang memiliki policy(), yaitu RefundablePayment. Ia mewarisi tepat satu default method, sehingga tidak ada ambiguitas dan Java langsung memakai implementasi tersebut (hasilnya “Refund maksimal 7 hari”).

2. Mengapa Java tidak memilih salah satu default secara otomatis?

CreditCardPayment mewarisi dua default method policy() dengan signature yang sama, satu dari RefundablePayment dan satu dari InstallmentPayment. Kedua interface itu tidak saling mewarisi, sehingga tidak ada yang lebih spesifik dibanding yang lain. Jika Java memilih sendiri, hasilnya bisa salah secara diam-diam: salah satu kebijakan akan hilang tanpa disadari programmer. Karena itu compiler menolak dan memaksa programmer memutuskan secara eksplisit. Di CreditCardPayment, konflik diselesaikan dengan meng-override policy() dan memanggil keduanya lewat RefundablePayment.super.policy() dan InstallmentPayment.super.policy(), dipisah dengan “; ”.

3b. T2

1. Mengapa ada interface dan abstract class, bukan salah satunya saja?

Keduanya punya peran yang berbeda. NotificationChannel (interface) adalah kontrak “bisa mengirim pesan”. NotificationService hanya perlu mengenal kontrak ini, sehingga kanal apa pun, termasuk anonymous class kanal LOG yang tidak punya induk, dapat didaftarkan. AbstractChannel (abstract class) menyediakan kode bersama yang membutuhkan state, yaitu penghitung sentCount, lastDelivered, dan urutan langkah send(...). Interface tidak bisa menyimpan field seperti itu. Jika hanya memakai interface, kode yang sama akan ditulis ulang di setiap kanal. Jika hanya memakai abstract class, setiap kanal terpaksa mewarisi class tertentu dan NotificationService terikat padanya.

2. Apa akibatnya jika send(...) tidak dibuat final?

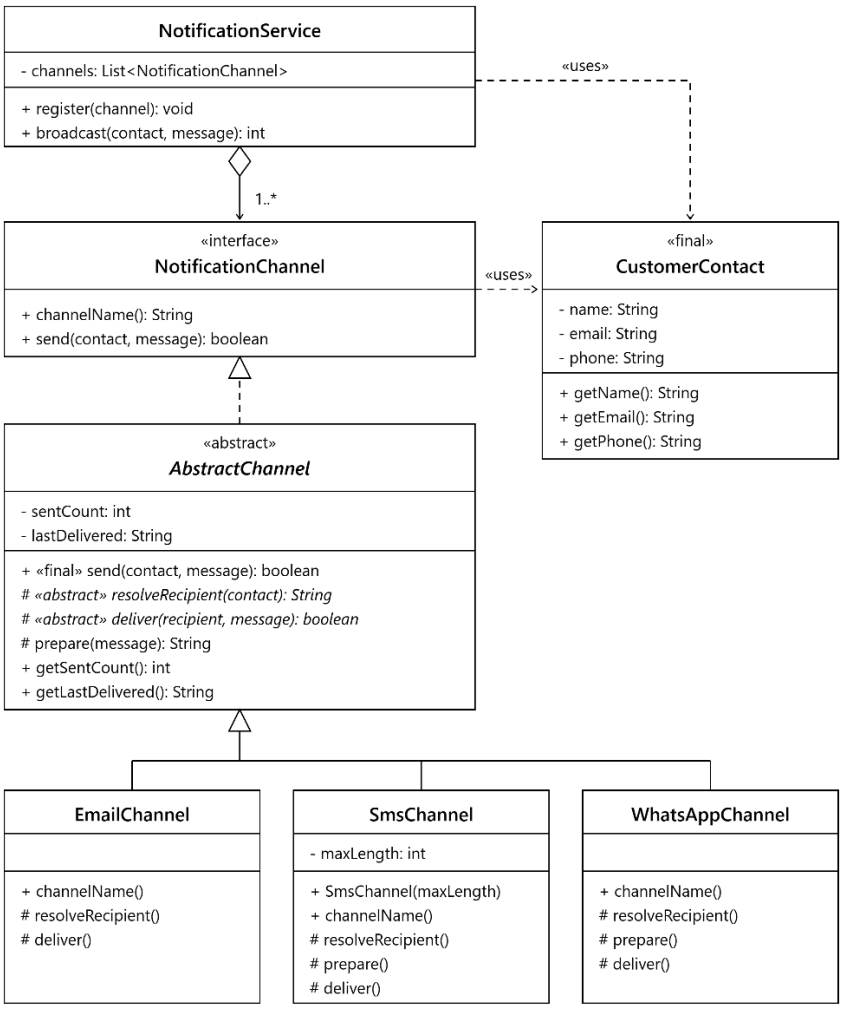
Subclass dapat meng-override seluruh alur pengiriman, misalnya melewati validasi pesan kosong, tidak memanggil resolveRecipient, atau lupa menaikkan sentCount dan mengisi lastDelivered. Akibatnya perilaku antarkanal menjadi tidak seragam dan jaminan urutan langkah (tolak pesan kosong, cari tujuan, siapkan pesan, kirim, lalu catat) hilang. Dengan final,

subclass hanya mengisi langkah-langkahnya lewat resolveRecipient, deliver, dan prepare. Pola ini adalah cikal bakal Template Method.

3. Mengapa NotificationService menyimpan List<NotificationChannel>, bukan List<AbstractChannel>?

Ini penerapan prinsip program to an interface. Jika list bertipe AbstractChannel, kanal LOG (anonymous class yang langsung mengimplementasikan NotificationChannel) tidak bisa didaftarkan, dan setiap kanal baru wajib mewarisi AbstractChannel. Dengan tipe interface, NotificationService tidak peduli bagaimana sebuah kanal dibuat. Kanal baru seperti Telegram atau push notification cukup ditambahkan sebagai class baru tanpa mengubah satu baris pun di NotificationService.

3c. Diagram UML



4. SelfCheck, TugasCheck dan TugasApp

SelfCheck

```
=== HASIL: 23 / 23 pemeriksaan lulus ===
PS D:\Praktikum_PBO>
```

TugasCheck

```
-- Parameter NIM
  [LULUS] P.1 Konstanta di StudentInfo sesuai NIM (3%, Rp4000, 108 karakter)

-- T1 - Evolusi API & diamond
  [LULUS] T1.1 PaymentMethod.adminFee(long) adalah default method; e-wallet = 0
  [LULUS] T1.2 Biaya admin kartu kredit & transfer sesuai parameter NIM
  [LULUS] T1.3 grandTotal(...) = total + biaya admin (kartu kredit, keranjang contoh, Ekspres)
  [LULUS] T1.4 Diamond policy(): kartu kredit menggabungkan kedua kebijakan

-- T2 - Sistem notifikasi
  [LULUS] T2.1 NotificationChannel interface; AbstractChannel abstract dengan send(...) final
  [LULUS] T2.2 EmailChannel, SmsChannel, WhatsAppChannel adalah subclass AbstractChannel
[SMS to 081234567890] xxxx
```